



ASTRALIS UI

The Analysis Report

Full library audit: verified build-level bugs, component defects and inconsistencies, competitive position (July 2026), and the prioritized roadmap.

7 July 2026

Companion to the Educational Report.

Generated from a full source, build & runtime audit

0. Method (what this report is based on)

- **Every source file** in `packages/astralis-ui/src` was read (components, const maps, hooks, theme, utils, build scripts) by five parallel deep-audit passes, one per category group.
 - **The library was built** (`vite build` + safelist + Tailwind compile) and the **compiled `dist/styles.css` was programmatically cross-checked**: every `astralis:*` class literal the source can emit (3,027 unique utilities) was tested for existence among the 14,294 compiled selectors, at base and at every breakpoint.
 - **Root causes were isolated with minimal reproductions** (e.g. a standalone Tailwind compile proving which theme namespace generates named `duration-*` utilities).
 - **The running library was inspected in a real browser** via the docs site (which consumes `dist/`, i.e. exactly what a consumer gets): DOM class output, computed styles, accent scoping.
 - Findings from the audit passes were **independently re-verified** before inclusion; several plausible-sounding claims were disproven and are listed in §1.4 so we don't chase ghosts later. Items I verified directly are marked **[confirmed]**; items reported by an audit pass that are consistent with the code but not re-executed by me are marked **[reported]**.
-

1. Systemic findings (build & architecture level)

These affect the whole library at once and are the highest-leverage fixes.

1.1 [confirmed] Named animation-duration tokens generate no CSS at all

Root cause: Tailwind v4 mints `duration-*` utilities from the `--transition-duration-*` theme namespace. Our `@theme` declares them under `--duration-*` (`--duration-fast`, `--duration-moderate`, ...). Wrong namespace → Tailwind generates nothing. Verified by minimal repro: `--duration-zippy` produced no utility; `--transition-duration-zippy2` produced one.

Impact: every `astralis:duration-fast` / `astralis:duration-moderate` in the library is a dead class. Affected: Card (`transition-all duration-moderate`), ThemeToggle, Tabs indicator (`duration-fast`), and anything else using named durations. They currently fall back to Tailwind's 150 ms default (confirmed via computed style in the browser: `transitionDuration: 0.15s`), so animations run — just never at the designed speed, and the tokens are decorative.

Fix: rename the `@theme` keys to `--transition-duration-*` (keep `--astralis-duration-*` as the var they point to), rebuild. One-file change.

1.2 [confirmed] 87 token-map entries emit classes that don't exist — silent no-op props

The coverage cross-check found 87 utilities referenced by the `src/const` maps with **no selector in the compiled CSS**. Groups:

Group	Examples	Why they don't compile
Logical-property sizing maps	<code>astralis:block-md</code> , <code>astralis:inline-8xl</code> , <code>astralis:min-block-xs</code> , <code>astralis:max-inline-prose</code> (~60 entries)	Tailwind v4 has no <code>block-*</code> / <code>inline-*</code> width-height logical utilities; the entire family is invented
<code>size-*</code> named sizes	<code>astralis:size-md</code> , <code>astralis:size-3xl</code> , <code>astralis:size-prose</code> , <code>astralis:size-vw</code> (~20)	<code>size-*</code> resolves from the spacing scale + fractions only; our named widths live in <code>--width-*</code> / <code>--height-*</code> , which <code>w-*</code> / <code>h-*</code> read but <code>size-*</code> does not
Stragglers	<code>astralis:max-w-8xl</code> , <code>astralis:max-w-auto</code> , <code>astralis:min-w-prose</code> , <code>astralis:w-prose</code> , <code>astralis:*-vw/vh</code> variants	Tokens (<code>prose</code> , <code>8xl</code> , <code>auto</code> , <code>vw</code>) missing from the namespaces those utilities actually read

Impact: any consumer setting the corresponding Box props (`boxSize`, `blockSize` / `inlineSize` families, `maxW="8xl"`, etc.) gets *nothing*, with no error anywhere. This is the worst failure mode the architecture allows.

Fix: three options per group — (a) delete the fabricated families from the maps and types (honest API), (b) define real `@utility` blocks for them in `tailwind-entry.css`, or (c) re-point map values at utilities that exist (e.g. `size-md` → `astralis:w-md` `astralis:h-md` — but see 1.3 first). Also: **make the coverage cross-check a CI step** — the script that found these should run on every build and fail on any miss. That single guardrail permanently closes the "silently doesn't compile" class of bug.

1.3 [confirmed] The safelist generator only sees the first class of multi-class values → responsive breakage

The regex in `gen-responsive-safelist.mjs` is `/["']astralis:(["'\s]+)["']/g` — it requires a quote ****immediately before**** `astralis:`. In a multi-class literal like

```
vertical: "astralis:h-full astralis:self-stretch astralis:border-l",
```

only `astralis:h-full` matches; the classes after each space are never safelisted. The compiled CSS proves it: `astralis:md:w-full` exists, but `astralis:md:border-l` and `astralis:md:-translate-x-1/2` **do not** (137 utilities lack breakpoint variants overall).

Impact: every responsive use of a multi-class token silently half-applies. Concrete: `<Separator orientation={{ base: "horizontal", md: "vertical" }} />` at `md` gets height/self-stretch but **no left border — an invisible separator**. Float's responsive `placement` loses its translate correction at breakpoints. (Most of the 137 are harmless — CVA-only classes that are never breakpoint-prefixed at runtime — but the token-map subset is real breakage.)

Fix: change the extraction to match every `astralis:`-prefixed token inside string literals (split candidates on whitespace — the same approach my verification script used), regenerate, rebuild. Five-line fix.

1.4 [confirmed] Disproven suspicions — these are fine (don't "fix" them later)

- Fraction classes (`astralis:w-1/2`, `astralis:md:w-1/2`) **do** compile. ✓
- Arbitrary values in component styles (`astralis:max-w-[calc(100vw-2rem)]`, `astralis:list-[circle]`, `astralis:active:scale-[0.98]`, `astralis:rounded-t-md`) **do** compile. ✓
- `astralis:inset-shadow-sm` (the `shadow="inner"` mapping) **exists** in the compiled CSS. ✓
- No `var()` inside any `@media` query in dist (the old breakpoint bug is fully dead). ✓
- Controlled-prop naming is **uniform** across all stateful components (`x/defaultX/onXChange`) — a genuine consistency win. ✓

1.5 [confirmed] `AstralisProvider tokens.brandColor` cannot recolor role-driven components

`generateBrandShades()` injects only the **primitive** shades (`--astralis-color-brand-50...900`) as inline styles on the provider div. But the *role* tokens (`--astralis-color-brand-solid`, the whole accent-channel default, `subtle/muted/emphasized/stroke/ring`) are declared at `:root`, where their `var(--astralis-color-brand-*)` references are substituted **at the root element** and inherited already-resolved. Overriding the primitives on a descendant does not re-run that substitution. Since virtually every component paints with role/accent tokens (see the live Button DOM: `bg-accent-solid`, `text-accent-label`), **the advertised brand-color API leaves most of the library in default yellow**. The docs hero works around exactly this by re-declaring all role tokens per theme — the provider must do the same (theme-aware, mirroring semantic.css formulas), or roles must stop being baked at `:root` (e.g. declare them on `.astralis` so the provider's inline override wins the cascade... note inline styles still lose to nothing here — re-declaration inside the provider style object is the robust fix).

1.6 [confirmed] Packaging & bundle-shape problems

Finding	Detail	Consequence
Blanket <code>"use client"</code>	One directive in <code>src/index.ts</code> , hoisted to the top of the single-chunk bundle	The entire library is client-code in RSC apps. Box, Text, Card, Table — pure presentational components that could server-render — all get shipped to the browser and hydrate
Single-chunk build, no <code>sideEffects</code> flag	<code>astralis-ui.es.js</code> is one 457 KB file; package.json lacks <code>"sideEffects": false</code>	Import one Button, ship everything — bundlers can't tree-shake within one module, and without the flag they won't even try
<code>exports.require</code> → UMD	<code>require</code> resolves <code>astralis-ui.umd.js</code> while a proper <code>astralis-ui.cjs.js</code> is built and never referenced	CJS consumers get a UMD wrapper; the CJS artifact is dead weight
<code>styles.css</code> weight	992 KB raw / ~117 KB gzip	The safelist force-generates sm/md/lg/xl variants of all 2,889 extracted utilities — including hundreds only ever used inside CVA variants that are never responsive (hover/focus/disabled states, arbitrary selectors). Easily halvable by safelisting only the const token maps (the only runtime-composed classes) instead of all of <code>src/components</code>
Metadata	<code>"private": true</code> (unpublishable), <code>"author": "Your Name"</code> , peer range <code>react ^19.1.1</code> only	Publishing blocked; React 18 consumers excluded by policy rather than necessity
[confirmed] Highlight TS4023	<code>HighlightCustomProps</code> isn't exported from its types module, so <code>vite-plugin-dts</code> errors twice per build (build still completes)	Declaration noise now; a hard failure if dts settings tighten

Recommended shape (this is what the strongest 2025-era libraries converge on): build with `preserveModules` (one output file per source module), per-file `"use client"` only where hooks/handlers exist, `"sideEffects": false` (CSS excepted), fix the exports map, and split theme CSS from utilities so consumers can theoretically load tokens without every utility.

1.7 On the "world shouldn't know it's Tailwind" goal — honest status

[confirmed] The DOM emits zero unprefixed Tailwind and zero raw hex/rgb from the token system; everything is `astralis:*` utilities + `astralis-accent-*` scopes + `--astralis-*` variables. Collision-proof and version-decoupled: goal achieved in the engineering sense. Not achieved (and not achievable with this approach): *recognizability* — anyone who knows Tailwind will read `astralis:inline-flex` fluently. If true opacity ever matters, the options are semantic classes generated per-recipe (Panda-style `astralis-button--solid`) or class-name hashing; both are major reworks with real DX costs and questionable payoff. Recommendation: accept prefix-level hiding as the design point and document it.

2. Component-level defects

Grouped by theme; each item carries file, severity, and confirmation status. **H/M/L** = high/med/low.

2.1 Correctness bugs

#	Sev	Where	Finding
C1	H	hooks/use-overlay-behavior.ts	[confirmed] Scroll-lock has no reference counting: each overlay saves <code>body.style.overflow</code> on open and restores on close. Open Modal → open Drawer (or nested modal) → close the <i>first</i> one ⇒ body scroll unlocks while the second is still open. Same stacking blindness in the focus trap (two document-level Tab handlers fight) and Escape (both overlays listen; one Esc can close more than intended). Fix: a tiny overlay-stack manager (lock counter + only-topmost-handles-Esc/Tab)
C2	H	data-entry/select, multi-select	[confirmed] No hidden <code><input></code> and no <code>name</code> prop → selected values never reach native form submission . Every peer library ships form integration; this blocks the most basic use case
C3	M	theme/provider.tsx	[confirmed] = \$1.5, listed here for the backlog: <code>tokens.brandColor</code> doesn't re-declare role tokens
C4	M	data-display/marquee/marquee-root.tsx	[confirmed] <code>autoFill</code> prop is accepted and silently ignored (<code>autoFill: _autoFill = false</code>). Ship it or remove it
C5	M	data-display/timeline/timeline-item.tsx	[reported] Indicator detection reads <code>child.type.displayName</code> via cast without first checking <code>typeof child.type !== "string"</code> ; a native element as first child can misbehave
C6	M	data-display/image	[reported] Load/error state can update after unmount when <code>src</code> changes mid-flight (no cancellation guard); error state may persist across <code>src</code> changes
C7	M	typography/highlight	[confirmed] <code>children</code> is typed <code>string</code> (TS users are safe) but JSX runtime accepts anything — a non-string child crashes <code>.split()</code> . Add a <code>typeof children === "string"</code> guard + graceful fallback
C8	L	data-display/avatar/avatar-group.tsx	[reported] <code>max={0}</code> (or negative) renders zero avatars plus a "+N" chip; clamp <code>max</code>
C9	L	data-entry/pin-input	[reported] After paste that fills all cells, focus lands on the last cell with its value selected — acceptable but re-typing overwrites; consider focusing first empty cell
C10	L	data-entry/slider	[reported] Multi-touch: always reads <code>touches[0]</code> ; pinch gestures can jump the thumb
C11	L	disclosure/carousel	[reported] <code>loopCount</code> relies on <code>animationiteration</code> events, which browsers fire unreliably under throttling; loop may not stop exactly on count (Marquee shares this pattern)

2.2 Theming / token purity (violations of our own system)

#	Sev	Where	Finding
T1	H	<code>navigation/tabs/tabs.styles.ts</code>	[confirmed] Tabs has no <code>colorScheme</code> prop at all — active state and the ink-bar indicator hardcode <code>brand-solid</code> . Accordion, Pagination, Steps, Carousel all take <code>colorScheme</code> ; Tabs is the odd one out. Port to the accent channel
T2	M	<code>disclosure/carousel/carousel.styles.ts</code>	[reported] Indicator/controls mix hardcoded <code>brand-*</code> with <code>accent-*</code> — <code>colorScheme</code> only partially respected
T3	M	<code>data-entry/switch/components/switch.tsx</code>	[confirmed] Thumb is <code>astralis:bg-white</code> — the only raw palette color in data-entry. Visually fine in dark mode but violates token purity; use a semantic (e.g. a dedicated <code>surface-panel</code> / white-on-any-theme token)
T4	M	<code>overlay/*/styles</code> , <code>image-lightbox</code>	[reported] Lightbox overlay hardcodes <code>rgba(0,0,0,0.85)</code> ; Marquee's gradient fallback hardcodes <code>#fff</code> and uses a malformed var name (<code>var(--astralis:surface-base)</code> — colon instead of dash, so the fallback always wins). QR defaults <code>#000</code> are acceptable (QR needs real contrast)
T5	L	<code>typography/code-block</code>	CodeBlock traffic lights hardcode <code>red/yellow/green-400</code> — arguably intentional skeuomorphism; document it as such or tokenize
T6	L	<code>const/color-mappings.ts</code>	[reported] The three big color maps lack <code>as const</code> (sibling maps have it) — weakens literal-type inference for <code>bg/color/borderColor</code> props

2.3 Accessibility

#	Sev	Where	Finding
A1	H	<code>data-display/marquee</code>	[confirmed] No <code>prefers-reduced-motion</code> handling anywhere in the library — Marquee animates unconditionally (WCAG 2.3.3). Carousel autoplay likewise. Add a media-query pause + <code>useReducedMotion</code> -style hook
A2	M	<code>data-display/calendar</code>	[reported] No arrow-key navigation across the date grid (the expected APG pattern for date pickers); cells are tab-only
A3	M	<code>data-entry</code> groups	[reported] <code>Radio.Group</code> (<code>role="radiogroup"</code>), <code>Checkbox.Group</code> , <code>CheckableTagGroup</code> render no <code>aria-label</code> / <code>aria-labelledby</code> plumbing
A4	M	<code>data-entry</code> × <code>field</code>	[reported] Inputs never wire <code>aria-describedby</code> to <code>Field.HelpText</code> / <code>Field.ErrorText</code> ids — the help/error text isn't announced with the control. Field already owns ids; this is cheap to close
A5	M	<code>disclosure/carousel</code>	[reported] Prev/Next control buttons ship icon-only with no default <code>aria-label</code> (indicators have them)
A6	M	<code>data-entry/pin-input</code>	[reported] Cells lack per-cell labels ("digit 1 of 4")
A7	L	<code>disclosure/pagination</code>	[reported] <code>aria-disabled</code> on the <code><nav></code> element is non-standard; belongs on the buttons
A8	L	<code>data-entry/select</code>	[reported] In-dropdown search inputs lack <code>aria-label</code> ; <code>required</code> from Field isn't reflected on the combobox trigger
A9	L	overlays	Radio/Select typeahead, Pagination/Carousel Home/End — nice-to-have keyboard depth beyond the (solid) existing roving-focus work

2.4 API consistency

#	Sev	Finding
I1	M	Size scales differ without reason: Badge has <code>xs-lg</code> , Tag <code>sm-lg</code> ; Button <code>xs-xl</code> ; most inputs <code>sm-lg</code> . Decide the canonical scale per component class (chip vs control) and align Tag/Badge
I2	M	Responsive props are inconsistent: Text/Box/layout are fully <code>Responsive<></code> ; Button/Badge/Tag scalar-only (<code>size</code> , <code>rounded</code>). Either is defensible; the mix isn't. Cheapest coherent rule: <i>style props responsive, component variants scalar</i> — then document it
I3	M	Ref targets vary: Select/MultiSelect forward the wrapper/trigger; inputs forward the <code><input></code> ; InputGroup forwards nothing (plain function). Document per component or standardize on "ref = the interactive element"
I4	L	Context conventions split: some compound parts throw outside root (Table, DataList), some default silently (Card). Pick one (throwing is safer)
I5	L	<code>displayName</code> coverage is patchy across compound parts (Accordion, Carousel, Pagination, overlay parts) — matters for DevTools and for code that sniffs <code>displayName</code> (see C5, which is also <i>fragile because of this</i>)
I6	L	Icon <code>size</code> accepts tokens <i>or</i> raw numbers; unique in the library — fine, but document the escape hatch

2.5 Duplication (refactor candidates, not bugs)

#	Where	What's duplicated
D1	<code>select</code> ↔ <code>multi-select</code>	<code>flattenOptions</code> , <code>isGroup</code> , option row rendering, portal + reposition wiring, spinner SVG — extract a shared <code>useSelectDropdown()</code> + <code><OptionList></code>
D2	<code>modal</code> ↔ <code>drawer</code>	Title/Description presence-flag effects, close buttons, backdrop, context shape — extract shared dialog primitives (would also be the natural home for the C1 overlay-stack fix)
D3	4+ typography components	Hand-rolled "split variant keys from HTML props" loops (Blockquote, Code, CodeBlock, List) — Box already derives this from map keys; share one <code>splitVariantProps(map, props)</code> util
D4	roving-focus keyboard handlers	Accordion / Tabs / (future Menu) each re-implement arrow-key roving — extract <code>useRovingFocus</code> before the Menu component lands (it'll need it anyway)
D5	status/check icons	Repeated inline SVGs (check, chevron, spinner, image-error) — a private <code>icons.ts</code> would end the drift

3. What we got right (worth saying plainly)

- **The token pyramid is genuinely good.** Primitive → semantic role → Tailwind bridge with `var()` indirection is the same architecture Chakra v3 and Radix Themes arrived at, and the per-palette role vocabulary (`solid/contrast/label/subtle/muted/emphasized/stroke/ring`) is disciplined and complete.
 - **The accent channel is the standout idea** — `colorScheme` at O(1) CSS cost is materially better than what Tailwind-based peers (daisyUI, HeroUI) do, and the live DOM confirms it works exactly as designed.
 - **Controlled/uncontrolled discipline is uniform** across all 15+ stateful components — rarer than it should be, even in big libraries.
 - **The responsive-prop engine with a single source of truth** (maps feed CVA + runtime + safelist + types) is a real architectural asset; the bugs found are in the *guardrails*, not the design.
 - **Interaction quality is above copy-paste-kit level:** grid-rows accordion animation, ResizeObserver ink-bar, roving tabindex done right, focus return on overlays, `inert` on closed panels, `useId` ARIA wiring throughout.
 - **Zero runtime styling dependencies** — no Radix/Floating-UI/framer-motion in the bundle; five hooks in ~400 lines carry all overlay behavior.
-

4. Competitive position

(Web-verified July 6–7 2026: npm registry, GitHub, official changelogs. Sources in the addendum.)

4.1 The landscape in one table

Library	Latest (Jul 2026)	Species	Styling	~Components	A11y foundation	npm/wk
Chakra UI	3.36 (v3, no v4)	styled	Emotion — still runtime CSS-in-JS	~110	Ark UI / Zag.js	1.65M
Material UI	v9 (Apr 2026; v8 skipped)	styled	Emotion (zero-runtime Pigment CSS on hold)	~60 + MUI X	in-house	9.1M
Mantine	v9 (Mar 2026, requires React ^19.2)	styled	CSS modules + CSS vars, zero runtime	120+ & 70 hooks	in-house	1.8M
HeroUI v3 (ex-NextUI)	3.2 (Mar 2026 rewrite)	styled	Tailwind v4 + decoupled <code>@heroui/styles</code> ; dropped framer-motion for CSS animations	75+ (+37 React Native)	React Aria Components	359k
Ant Design	v6 (Nov 2025)	styled	dropped CSS-in-JS for CSS variables	~72	in-house (weakest of majors)	3.2M
daisyUI	v5	pure-CSS plugin	Tailwind 4 plugin, zero JS	68	none (CSS only)	758k
shadcn/ui	CLI v4, Vercel-owned	copy-paste registry	Tailwind v4 + CVA + <code>data-slot</code>	64 registry	Base UI (default since Jul 2026) / Radix	10M+ registry/wk
Radix Primitives	unified <code>radix-ui</code> pkg	headless	none	30	own — but creators left to build Base UI; Themes slow, Colors frozen	9.2M
Base UI	v1.0 Dec 2025 , now 1.6	headless	none; <code>render</code> prop	37+	Radix+Floating-UI+MUI veterans	6.5M
Ark UI	v5 (Park UI stale)	headless	none (Zag.js machines, multi-framework)	~45	Zag.js	915k
React Aria Components	1.19 (Jun 2026)	headless	none	~55	deepest (Adobe, screen-reader tested, 30+ locales)	2.9M

4.2 Where Astralis stands

Bets the market validated:

- **Zero-runtime CSS is now the consensus** — Mantine, Ant Design v6, HeroUI v3 and daisyUI all moved there; the two runtime-CSS-in-JS holdouts (Chakra on Emotion, MUI with Pigment stalled) are trying to leave. Astralis was born on the right side of this shift, and Chakra — the library with the most similar API surface (compound components, semantic tokens, `colorPalette` $\approx$ our accent channel) — is still paying the Emotion tax we don't.
- **React-19-first** is no longer aggressive (Mantine requires 19.2; HeroUI and Base UI require 19).
- **The accent channel** matches Chakra v3's `colorPalette` mechanism at lower CSS cost than any Tailwind-based peer, and our precompiled-CSS install story is simpler than HeroUI/shadcn, which require consumers to run Tailwind themselves.
- Even our quirky picks were trend-right: Marquee, QRCode, Timeline, Stat and a standalone Calendar all got added by Chakra/Mantine/Ark in 2025–26. ThemeToggle as a component is nearly unique to us.

Below the bar, fixable:

1. **Behavioral a11y depth — now a legal matter, not just polish.** The European Accessibility Act has been enforced since June 2025 (first lawsuits filed); buyers now audit vendor component libraries. The headless engines (Zag, React Aria, Base UI) encode thousands of person-hours of screen-reader edge cases our five hooks don't. Owning our own behavior is the differentiator *and* the liability: we must close §2.3 **and publish per-component APG conformance docs** (keyboard tables, ARIA notes — the React Aria/Base UI bar), or "no headless dependency" reads as risk.
2. **Form integration** — every styled peer submits with native forms (C2).
3. **The Feedback/Menu hole** — Toast and Menu ship in **11 of 11** surveyed libraries; Progress 11/11, Collapsible & ToggleGroup 10/11, Combobox 9/11, Textarea & NumberInput & Alert & Breadcrumb 8/11. We ship none of them (§5).
4. **RSC packaging** — the accepted norm is per-file `"use client"` banners preserved through bundling, ESM `exports` maps, `sideEffects: false`; we ship one blanket client chunk (§1.6). (Nobody ships true server components from a styled library — parity, not leadership, is the goal here.)
5. **AI-agent surface** — `llms.txt` + `.md` doc endpoints + an MCP server are shipped by shadcn, Chakra, Mantine, HeroUI, Base UI, Ark, daisyUI and React Aria. The holdouts are MUI, Radix and Tailwind. For a young library this is the main defense against shadcn's training-data dominance: agents fetch current props instead of hallucinating.

The unclaimed opportunity (strategic): as of Firefox 147 (Jan 2026), the **native overlay platform is complete in every engine** — the `popover` attribute, CSS anchor positioning, `<dialog>.showModal()`, and `@starting-style` — and **no major library uses it** (React Aria's popover is actively incompatible with the native one; Base UI/Radix still ship Floating UI + portal divs + JS focus traps, partly because their architectures predate it). Astralis is uniquely unencumbered: our overlays already sit on five swappable in-house hooks. A CSS-native overlay stack (with OddBird's ~8 KB anchor-positioning polyfill for the browser tail) + OKLCH relative-color brand generation is a coherent "CSS-native, zero-runtime" story no competitor currently tells — and it would *shrink* our code, not grow it.

5. Roadmap: components to add

The "n/11" figures are how many of eleven surveyed libraries (Chakra, MUI, Mantine, HeroUI, AntD, daisyUI, shaden, Radix, Base UI, Ark, React Aria Components) ship the component — verified against their live component lists, July 2026.

5.1 New category: Feedback (the glaring hole — six of the twelve most-universal gaps live here)

Component	Ships in	Notes
Toast	11/11	The single most conspicuous absence. Needs a manager (queue, portal, timers, swipe-dismiss, <code>role="status"</code>) + imperative API + <code><Toaster></code> mount; the overlay-stack work from C1 is a shared prerequisite
Progress (linear + circular)	11/11	<code>aria-valuenow</code> plumbing mirrors Slider's
Alert / Callout	8/11 (100% of styled libs)	Nearly free: <code>surface/label/stroke-{status}</code> tokens already exist and are underused
Skeleton	7/11 (all styled libs)	<code>animate-pulse</code> is already compiled
Spinner	7/11	Extract the one already inside Button/Select (D5)
EmptyState	4/11	Cheap, and pairs naturally with Table/DataList docs

5.2 New category: Menus (gap #2 — Menu ships in 11/11)

One **Menu** primitive powering the whole cluster — exactly how Base UI/Radix/Ark factor it:

- **DropdownMenu** first (anchor positioning + roving focus + typeahead — builds directly on `useAnchorPosition` + the D4 `useRovingFocus` extraction)
- **ContextMenu** as a trigger variant of the same engine (the Chakra/Ark pattern, not a separate component)
- **Menubar** later; **Command palette** eventually (2/11 but high wow-factor; pairs with docs search)

5.3 Fill existing categories

Category	Additions (in priority order)
Data entry	Textarea 8/11 (promote <code>Input.TextArea</code> to a first-class documented component — its absence reads as an oversight), NumberInput 8/11, Combobox/Autocomplete 9/11 (Select+search is 80% there), DatePicker 7/11 (input + popover over the existing Calendar — the Calendar was clearly built for this; Chakra just added theirs), FileUpload/Dropzone 7/11, Rating 6/11, PasswordInput (document <code>Input.Password</code> as its own page), TagsInput 3/11 (MultiSelect sibling), ColorPicker 6/11 (later). RangeSlider: keep as a multi-thumb Slider feature, not a separate component (the Chakra/Ark/MUI/RAC pattern — we already match it)
Navigation	Breadcrumb 8/11 (easy win), Link 7/11 (we ship <i>no Link at all</i>), NavigationMenu 3/11 (later)
Disclosure	Collapsible 10/11 (extract Accordion's grid-rows mechanic as a standalone), Toggle/ToggleGroup 10/11, SegmentedControl (Tabs <code>segmented</code> variant already proves the visual)

Category	Additions (in priority order)
Data display	Kbd 5/11 (pairs with Code), Clipboard/CopyButton 3/11 (docs already hand-rolled one — dogfood it back into the library), Tree/TreeView 5/11 (later)
Overlay	AlertDialog (Modal preset with <code>role="alertdialog"</code> + initial-focus-on-cancel), HoverCard 6/11, ScrollArea 5/11 (later), Splitter/Resizable 5/11 (later)
Utilities (new docs section)	VisuallyHidden , Portal (both already exist internally — export them), Presence, FocusTrap

Skip unless strategic: Chart, DataGrid, Transfer, Cascader, Tour, Watermark, Mentions, rich-text editor — the enterprise tail that AntD/Mantine own.

Sequencing: (1) the Feedback drop (Alert, Spinner, Skeleton, Progress, then Toast) + Menu closes ~8 of the 12 universal gaps in two coherent releases; (2) Textarea + NumberInput + Combobox complete Data entry; (3) Breadcrumb + Link + Kbd are ≤ 1 -day wins to interleave; (4) DatePicker rides the existing Calendar.

6. Modern platform & React features to adopt

Ordered by readiness (verdicts verified July 2026):

1. **React 19 `ref` as prop — drop `forwardRef`** (*stable; `forwardRef` is headed for deprecation; we already require React 19*). Deletes the polymorphic-cast pain in §9 of the educational report, the `as any` `displayName` casts, and ~50 wrappers. Mechanical, codemod-able. While in there: `<Context>` as provider (drop `.Provider`) and `use()` for conditional context reads.
2. **React 19.2 additions** (*stable since Oct 2025*): `<Activity>` — hidden-but-state-preserved subtrees, directly relevant to Tabs/Carousel `keepMounted` panels — and `useEffectEvent`, which cleans up exactly the stale-closure-in-listener patterns Slider/Marquee hand-roll with refs. Also on the form side: `useFormStatus`-aware submit Buttons and `<form action>` support in Field/Input are the expected React 19 integration points once C2 lands.
3. **CSS `@starting-style` + `transition-behavior: allow-discrete`** (*Baseline Aug 2024*). Enter/exit transitions from `display:none` with zero JS — retires the double-RAF dance in `usePresence`. Worth copying Base UI's complementary pattern too: expose `[data-starting-style]` / `[data-ending-style]` attributes so consumer transitions work everywhere.
4. **`popover` attribute + native `<dialog>`** (*Baseline; Invoker Commands `commandfor` hit Baseline Jan 2026*). Top-layer rendering ends z-index management and portal divs; `showModal()` gives focus trap, `::backdrop`, and Esc natively — which is the C1 overlay-stack fix, done by the platform. Notably **no major library renders a real `<dialog>` or native popover** (React Aria's is actively incompatible) — see §4.2's "unclaimed opportunity". Caveat: `closedby` / `popover="hint"` lack Safari; progressive enhancement only.
5. **CSS Anchor Positioning** (*in every engine as of Firefox 147, Jan 2026; ~83–91% traffic; Interop-2026 edge cases remain*). Replaces `useAnchorPosition` with the OddBird polyfill (~8 KB, feature-detected) covering the tail — smaller than shipping Floating UI to everyone. Design Popover/Tooltip/Select so the positioning engine is swappable, then swap.
6. **OKLCH + `color-mix()` + relative color syntax for brand shades** (*all Baseline 2023–24; Tailwind v4's own palette is OKLCH*). Our RGB-blend `generateBrandShades` washes out saturated hues at the ramp ends; the standard recipe is "vary lightness, hold chroma/hue" in OKLCH. Moving generation into CSS (`oklch(from var(--brand) ...)` / `color-mix(in oklch, ...)` behind `@supports`) makes `brandColor` a one-variable API — which *also* solves §1.5 structurally.
7. **Container size queries** (*Baseline Feb 2023, >93%*). `@container` variants for Card/Stat/DataList ("responsive to parent, not viewport") — a differentiator most styled libraries still lack. (Style queries: no Firefox — hold.)
8. **`data-slot` attributes** (*the de facto Tailwind-v4 styling contract, shaden-popularized*). Stamp compound parts (`data-slot="card-header"`) so consumers can target internals without us exporting class names. Cheap, high goodwill.
9. **`light-dark()`** (*Baseline, but the wrong primary tool for a themable library — colors only, no manual override*). Keep class-based dark mode; do set `color-scheme` so native controls and scrollbars adapt. Adopt opportunistically.

10. **CVA vs tailwind-variants** (*watch item*). CVA 0.7.1 is ~2 years untouched with its 1.0 beta stalled — effectively maintenance mode, though stable and ubiquitous. **tailwind-variants v3** is the actively developed alternative with first-class slots, which would map neatly onto our compound components. No urgency; evaluate when compound styling next hurts.
 11. **View Transitions API** (*same-document Baseline Oct 2025; React's `<ViewTransition>` still canary*). Don't depend on it yet; just avoid squatting on `view-transition-name`.
 12. **Docs modernization** (*the 2026 table stakes, per §4.2.5*): `llms.txt` + `llms-full.txt` + `.md` suffix on every docs URL; an **official MCP server** (minimum viable: `list_components` / `get_props` / `get_example` / `search_docs` — Mantine's is backed by static published data, cheap to run); per-component **APG conformance pages** (keyboard tables + ARIA notes — doubly important post-EAA given our self-built hooks); a **theme builder** that exports CSS variables *and* an agent-consumable preset (shadcn's `/create` pattern) — our runtime brand generation is a natural fit; Algolia DocSearch v4 (free OSS tier now bundles conversational "Ask AI"). Skip: heavy embedded sandboxes, public Storybook.
-

7. The prioritized backlog (single list, do in order)

Po — silent breakage & core promises

1. Fix duration namespace (§1.1) — 1 file
2. Fix safelist regex (§1.3) — 5 lines
3. Purge/replace the 87 dead map entries (§1.2) + add the CSS coverage check to CI
4. Provider `brandColor` role re-declaration (§1.5)
5. Overlay stack manager: scroll-lock refcount, topmost-only Esc, trap coordination (C1)
6. Select/MultiSelect hidden inputs + `name` (C2)

P1 — coherence & a11y 7. Tabs `colorScheme` via accent channel (T1); Carousel accent completion (T2); Switch thumb token (T3); Marquee var typo (T4) 8. `prefers-reduced-motion` support (A1); `aria-describedby` Field wiring (A4); group labels (A3); Calendar grid arrows (A2); Carousel control labels (A5) 9. Packaging: preserveModules + per-file "use client" + sideEffects + exports fix + metadata (§1.6); Highlight TS4023 (export the type) 10. Tag/Badge scale alignment (I1); responsive-prop policy (I2); Highlight runtime guard (C7); Marquee `autoFill` decision (C4)

P2 — leverage & growth 11. Shared extractions: `useRovingFocus`, dialog primitives, select-dropdown internals, `splitVariantProps`, icon set (D1–D5) 12. Feedback category (Toast last), then Menus, then data-entry fills (§5) 13. React 19 ref-as-prop migration; `@starting-style`/popover/dialog adoption (subsumes the C1 fix long-term); OKLCH shade generation (§6) 14. CSS diet: safelist only token maps, measure, iterate (§1.6) 15. Docs: llms.txt + `.md` endpoints, MCP server, per-component APG conformance pages, theme builder (§6.12)

Addendum: research sources (verified July 6–7, 2026)

- Chakra v3 announcement & Q1-2026 recap — chakra-ui.com/blog/announcing-v3, chakra-ui.com/blog/q1-2026-recap
- MUI v9 — mui.com/blog/introducing-mui-v9 · Pigment CSS on hold — github.com/mui/pigment-css
- Mantine v9 changelog (React ^19.2, MCP server, agent skills) — mantine.dev/changelog/9-0-0
- HeroUI v3 rewrite (React Aria + Tailwind v4, framer-motion dropped) — heroui.com/docs/react/releases/v3-0-0, infoq.com/news/2026/07/heroui-v3-rewrite
- Ant Design v6 (CSS variables, React 19 native) — dev.to/zombiej/ant-design-60-is-released-bfa
- daisyUI v5 + 2025 Wrapped — daisyui.com/docs/v5, daisyui.com/blog/daisyui-2025-wrapped
- shadcn/ui: Tailwind v4 + data-slot (Feb 2025), MCP server (Aug 2025), Base UI default (Jul 2026) — ui.shadcn.com/docs/changelog
- Base UI v1.0 (Dec 11, 2025) & releases — base-ui.com/react/overview/releases, infoq.com/news/2026/02/baseui-v1-accessible · render prop / useRender — base-ui.com/react/utils/use-render · animation attributes — base-ui.com/react/handbook/animation
- Radix status (unified package, post-WorkOS trajectory) — radix-ui.com/primitives/docs/overview/releases
- Ark UI — ark-ui.com · Park UI preset staleness — npmjs.com/package/@park-ui/panda-preset
- React Aria Components 1.19 — react-aria.adobe.com · native-popover incompatibility — github.com/adobe/react-spectrum/issues/7067
- Tailwind v4 OKLCH palette & color-mix — tailwindcss.com/blog/tailwindcss-v4
- CSS anchor positioning in Firefox 147 — developer.mozilla.org/en-US/docs/Mozilla/Firefox/Releases/147 · polyfill — github.com/oddbird/css-anchor-positioning
- React 19 (ref as prop) — react.dev/blog/2024/12/05/react-19 · React 19.2 (`<Activity>`, `useEffectEvent`) — react.dev/blog/2025/10/01/react-19-2
- European Accessibility Act enforcement — levelaccess.com/compliance-overview/european-accessibility-act-eaa
- llms.txt spec & adoption analysis — llmstxt.org, limy.ai/blog/llms.txt-in-2026-the-full-guide
- Algolia DocSearch v4 with Ask AI — algolia.com/blog/product/docsearch-reimagined
- tailwind-variants v3 — github.com/heroui-inc/tailwind-variants/releases
- Debunked during verification: "Chakra v4 Tailwind rewrite" (spam-domain origin) and "HeroUI acquired by LambdaTest" (no evidence) — flagged so they don't re-enter our planning.